

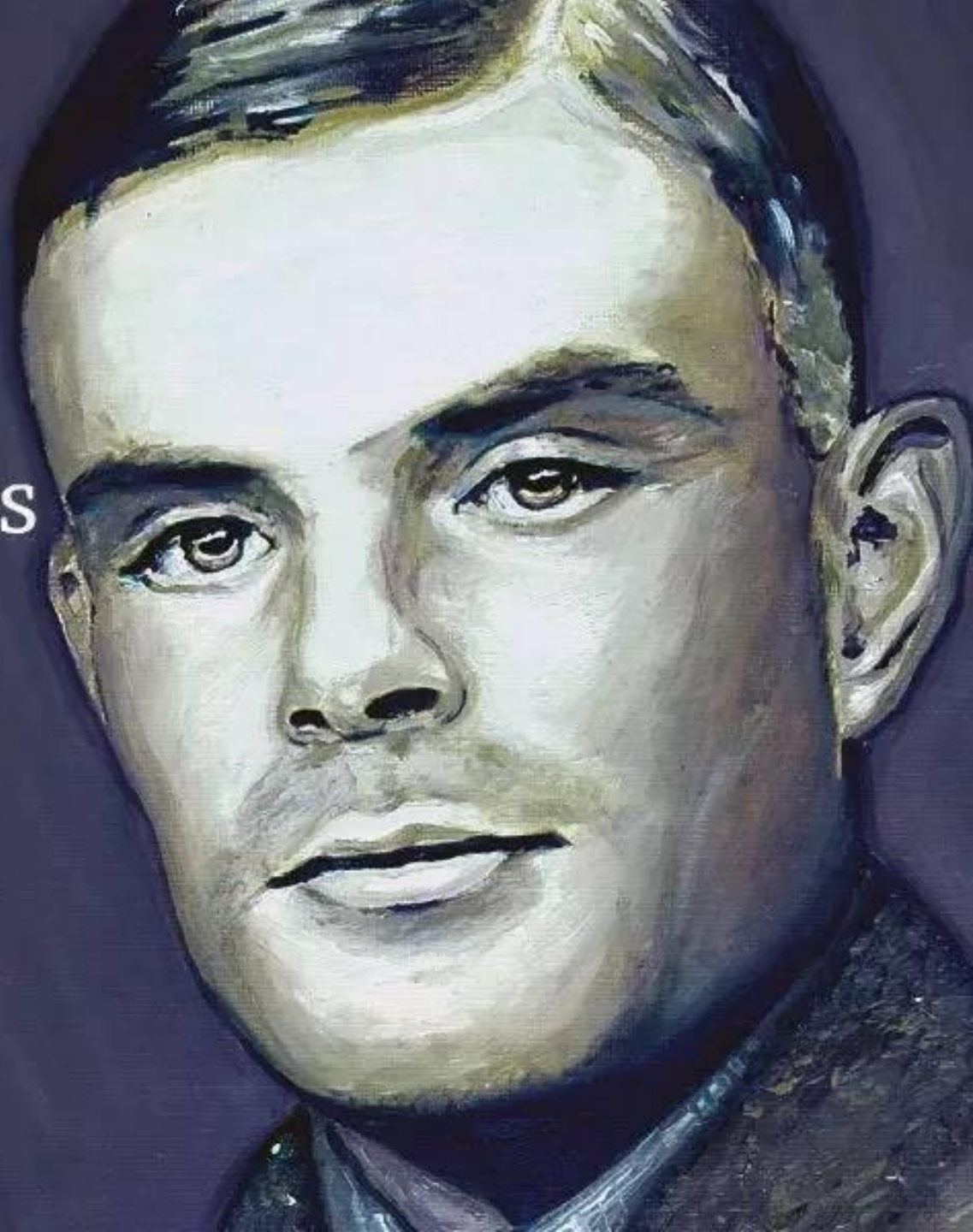
Algorithm Design & Analysis

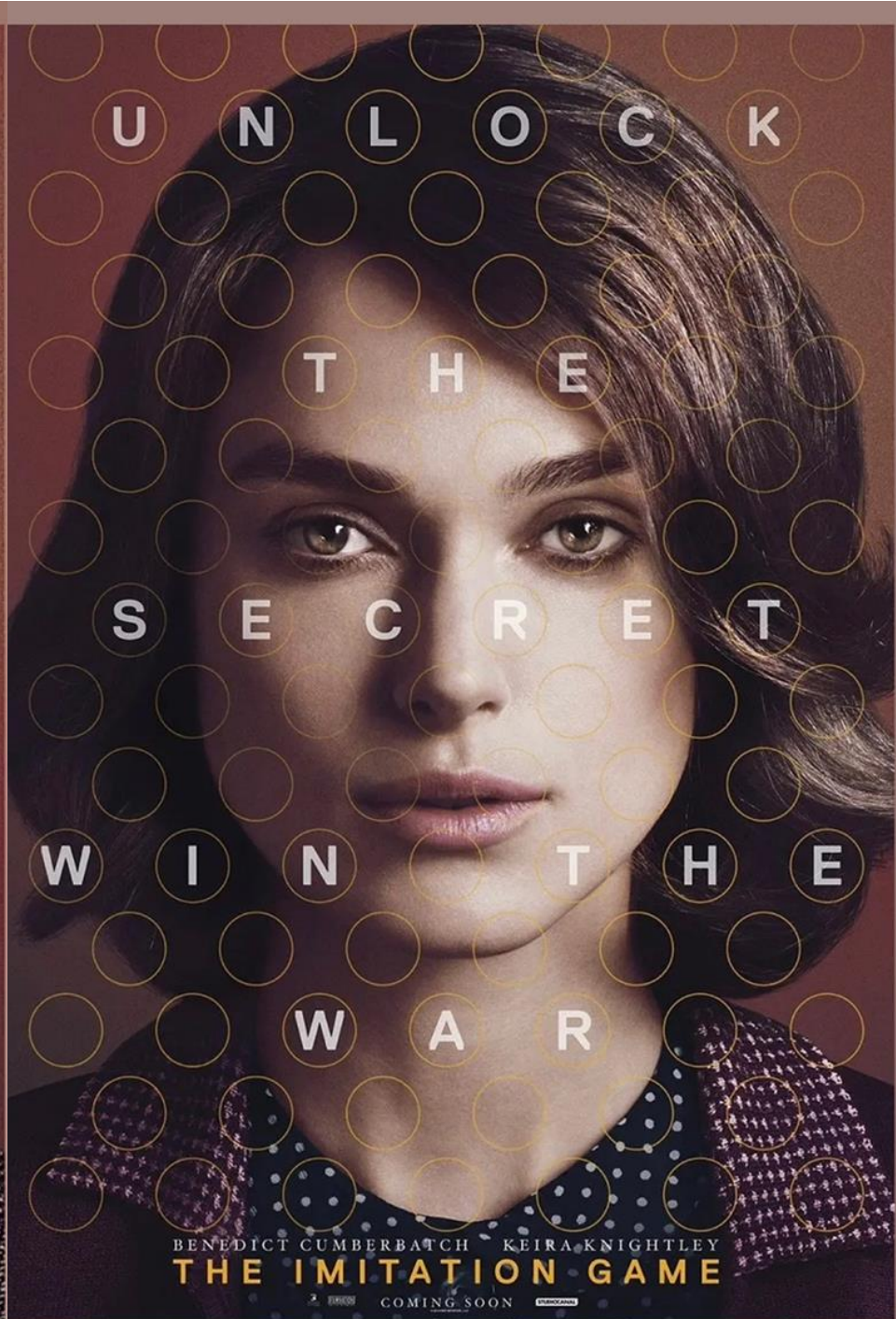
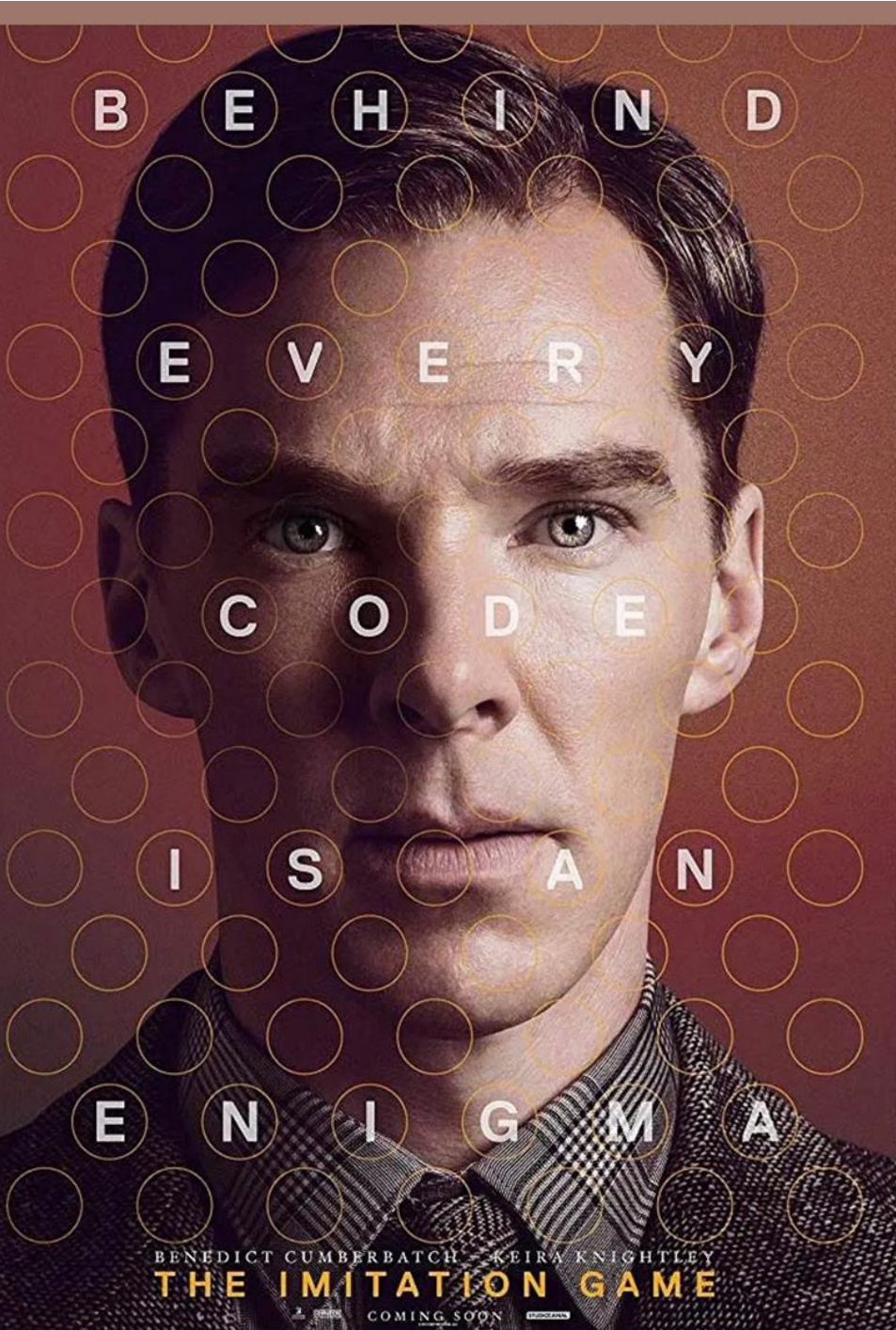
Lecture 2

SWUFE

“Sometimes
it's the people
no one imagines
anything
of who do
the things
that no
one can
imagine.”

Alan Turing







Review

算法设计与分析

- 算法背景

- 算法实例

- 调度问题、投资问题

$$\frac{(m+n-1)!}{m!(n-1)!} = \Omega((1 + \epsilon)^{m+n-1})$$

- 常见算法

- 快速排序、插入排序、冒泡排序、二分归并



一、算法的复杂性问题

二、算法及其时间复杂度

三、算法的伪码描述

四、函数的渐近的界



一、算法的复杂性问题

二、算法及其时间复杂度

三、算法的伪码描述

四、函数的渐近的界



一、算法复杂性问题

01

复杂性实例

02

NP-hard



一、算法复杂性问题

01

复杂性实例

02

NP-hard

复杂性实例

货郎问题

货郎建模

0-1问题

0-1建模

双机调度

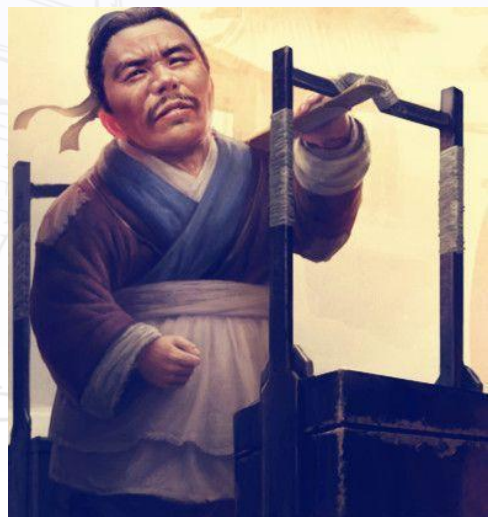
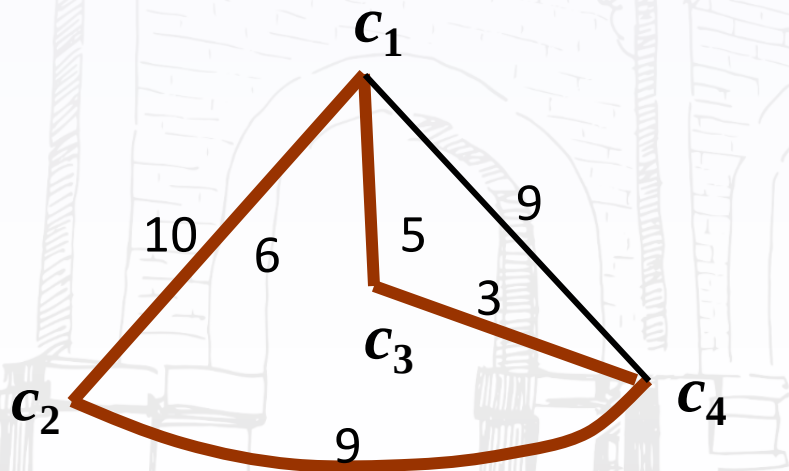
双机建模

几个例子

货郎问题/邮政问题（有权图）--哈密顿（无权图）

问题：有 n 个城市，已知任两个城市之间的距离.求一条每个城市恰好经过1次的回路，使得总长度最小。

求：一个最短的路程方案



复杂性实例

货郎问题

货郎建模

0-1问题

0-1建模

双机调度

双机建模

输入： 有穷个城市的集合 $C = \{c_1, c_2, \dots, c_m\}$, 距离

$$d(c_i, c_j) = d(c_j, c_i) \in \mathbb{Z}^+, \quad 1 \leq i < j \leq m$$

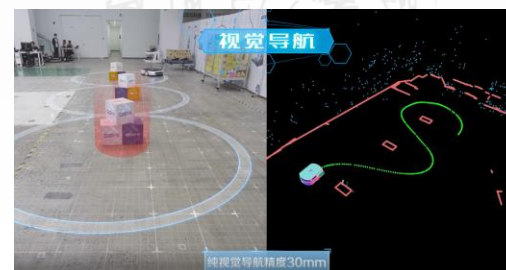
解： $1, 2, \dots, m$ 的排列 $k_1, k_2, \dots, k_m$ 使得

$$\min \left\{ \sum_{i=1}^{m-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_m}, c_{k_1}) \right\}$$

现状： 至今无有效的算法 存在大量问题与它难度等价

问题： 是否存在近似算法？ 如何处理这类问题？

应用：



复杂性实例

货郎问题

货郎建模

0-1问题

0-1建模

双机调度

双机建模

0-1背包问题

0-1背包问题：有 n 个物品要装入背包，第 i 件物品的重量 w_i 价值 $v_i, i = 1, 2, \dots, n$. 背包最多允许装入的重量为 B , 问如何选择装入的物品，使得总价值达到最大？

实例： $n = 4, B = 6$, 物品的重量和价值如下

标号	1	2	3	4
重量 w_i	3	4	5	2
价值 v_i	7	9	9	2

\$12
2kg

\$20
3kg

\$10
1kg

\$15
2kg





复杂性实例

货郎问题

货郎建模

0-1问题

0-1建模

双机调度

双机建模

0-1背包问题建模

问题的解：0-1向量 $\langle x_1, x_2, \dots, x_n \rangle$

$x_i \Leftrightarrow$ 物品 i 装入背包

目标函数

$$\max \sum_{i=1}^n v_i x_i$$

约束条件

$$\sum_{i=1}^n w_i x_i \leq B$$

$$x_i = 0, 1, i = 1, 2, \dots, n$$

应用：电商商品推荐（价格-评分）最大化评分

复杂性实例

货郎问题

货郎建模

0-1问题

0-1建模

双机调度

双机建模

双机调度问题



问题的解：有 n 项任务，任务 i 的加工时间为 $t_i, t_i \in \mathbb{Z}^+, i = 1, 2, \dots, n$. 用两台相同的机器加工，从0时刻开始计时，完成时间是后停止加工机器的停机时间.问如何把这些任务分配到两台机器上，使得完成时间达到最小？

实例：任务集 $S = \{1, 2, 3, 4, 5, 6\}$,
 $t_1 = 3, t_2 = 10, t_3 = 6, t_4 = 2, t_5 = 1, t_6 = 7$

解：Machine 1的任务：1, 2, 4
Machine 2的任务：3, 5, 6



有没有别的
更好的算法？

完成时间： $\max\{3+10+2, 6+1+7\}=15$



复杂性实例

货郎问题

货郎建模

0-1问题

0-1建模

双机调度

双机建模

双机调度问题建模

解： 0-1向量 $\langle x_1, x_2, x_3 \dots x_n \rangle$, $x_i = 1$ 表示任务 $i=1, 2, \dots, n$.

不妨设机器1的加工时间 $\leq$ 机器2的加工时间
令 $T = t_1 + t_2 \dots + t_n$, $D = \lfloor T/2 \rfloor$, Machine 1 的加工时间不超过 D , 且达到最大.



如何对该问题建模？目标函数与约束条件是什么？



一、算法复杂性问题

01

复杂性实例

02

NP-hard

NP-Hard

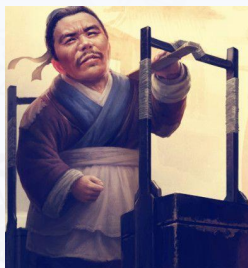
NP简介

算法目标

引出重要性

7大问题

NP-hard问题



\$12
2kg

\$10
1kg



\$20
3kg

\$15
2kg



- 上述这样的问题有数千个
- 至今没找到有效算法，所有算法均是**非多项式**的
- 也没有人能证明这类难题**不存在**多项式时间的算法.
- 从是否存在多项式时间算法的角度看，**这些问题**彼此是**等价**的.这些问题的难度处于可有效计算的边界.

NP-Hard

NP简介

算法目标

引出重要性

7大问题

NP、NPC、NP-hard问题

P问题 多项式时间问题

NP问题

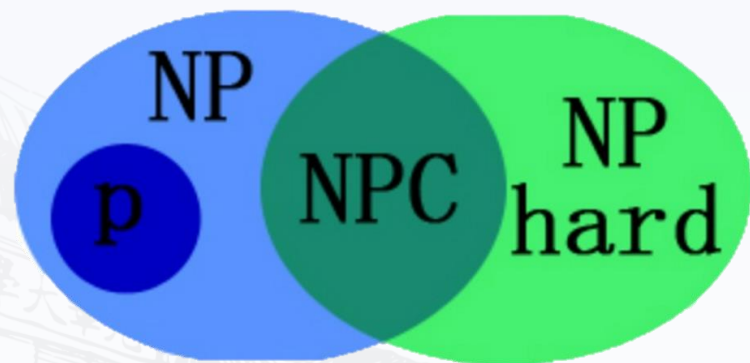
不能在多项式时间解决，但可在多项式时间内得到验证

NPC问题

是一个NP问题，所有NP问题都可以规约到该问题,意味着如果一个NPC问题可以被解决，那么所有对应的NPC问题都可以被解决。（超难）

NP-hard问题

NP-Hard问题要比 NPC问题的范围广，NP-Hard问题没有限定属于NP。如果一个问题被证明是NP-Hard的，那么它不一定是NP问题，但它至少与所有NP问题一样难以解决





NP-Hard

NP简介

算法目标

引出重要性

7大问题

Algorithm + Data Structure =Programming

好的算法关注的问题 $\Leftrightarrow$

- 1.提高求解问题的效率
- 2.节省存储空间

算法的研究目标

- 问 题 $\rightarrow$ 建模并寻找算法
- 算 法 $\rightarrow$ 算法的评价
- 算法类 $\rightarrow$ 问题复杂度估计
- 问题类 $\rightarrow$ 能够求解的边界

1.算法设计技术

2.算法分析方法（定量）

3.问题复杂度分析（ $N^2, N^3, \log N \dots$ ）

4.计算复杂性理论（P/NP/NPC）



NP-Hard

NP简介

算法目标

引出重要性

7大问题

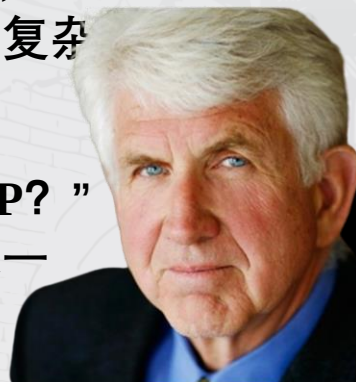
NP-hard问题引出算法重要性

- 算法设计与分析技术在计算机科学技术领域有着重要的应用背景
- 算法设计分析与计算机复杂性理论研究是计算机科学技术的核心研究领域
- 1966-2022，图灵奖中56次颁奖中，11人以算法设计，8人以计算理论和复杂性研究领域的杰出贡献获奖
- 计算复杂性理论的核心课题“ $P=NP?$ ”是本世纪7个最重要的数学问题之一



AVI
WIGDERSON

艾维·维格森



BOB METCALFE

(2022年图灵奖获得者) 以太网发明者



JACK DONGARRA

(2021年图灵奖获得者) 通过对线性代数运算的高效数值算法、并行计算编程机制和性能评估工具的贡献，引领了高性能计算的世界



NP-Hard

NP简介

算法目标

引出重要性

7大问题

本世纪7大数学问题

1

P问题对NP问题

2

霍奇(Hodge)猜想

3

庞加莱(Poincare)猜想
已解决

4

黎曼(Riemann)假设

5

杨-米尔斯(Yang-Mills)
存在性和质量缺口

6

纳维叶-斯托克斯(Navier-Stokes)方程的存在性与光滑性

7

贝赫(Birch)和斯维纳通(Swinnerton-Dyer)猜想

2000年，美国克莱数学研究所公布了**世界七大数学难题**，
又称千年大奖问题，规定对每一难题的破解者颁发**一百万美元**的
奖金。其中P与NP问题被列为这七大数学难题之首。



小结

- NP-hard问题的几个例子：
 - 货郎问题
 - 0-1背包问题
 - 双机调度问题
- NP-hard问题的计算现状
- 计算复杂性理论的核心—NP-完全理论
- 算法研究的主要内容及其重要意义



一、算法的复杂性问题

二、算法及其时间复杂度

三、算法的伪码描述

四、函数的渐近的界





算法与复杂度

问题及实例

算法

基本运算与规模

输入规模

基本运算

复杂度

平均复杂度

检索例子

顺序检索

最坏情况

平均情况

改进检索例子

时间估计

问题及实例

- **问题**
需要回答的一般性问题，通常包含若干参数
- **问题描述**
 - 定义问题参数（集合、变量、函数、序列等）
说明每个参数的取值范围及参数间的关系
 - 定义问题的解
说明解满足的条件（优化目标或约束条件）
- **问题实例**
参数的一组赋值可得到问题的一个实例（排序长度为5）



算法与复杂度

问题及实例

算法

基本运算与规模

输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

算法

• 算法

有限条指令的序列

这个指令序列确定了解决某个问题的一系列运算或操作

• 算法A解决问题P

把问题P的任何实例作为算法A的输入

每步计算是确定性的

A能够在有限步骤内停机

输出该实例的正确的解



基本运算与输入规模

- **算法时间复杂度**：针对制定**基本运算**，计算算法所作运算次数
- **基本运算**：比较、加法、乘法、置指针、交换...
- **输入规模**：输入串编码长度
通常用下述参数度量：数组元素多少，调度问题的任务数量，图的顶点个数与边数等。
- **算法基本运算次数可表为输入规模的函数**
- **给定问题和基本运算就决定了一个算法类**

算法与复杂度

问题及实例

算法

基本运算与规模

输入规模

基本运算

复杂度

平均复杂度

检索例子

顺序检索

最坏情况

平均情况

改进检索例子

时间估计



算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

输入规模

- 排序：数组中元素的个数 n
- 检索：被检索数组的元素个数 n
- 整数乘法：两个整数的位数 m, n
- 矩阵相乘：矩阵的行列数 i, j, k
- 图的遍历：图的顶点数 n , 边数 m

.....



算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

基本运算

- 排序：元素之间的**比较**
- 检索：被检索元素x与数组元素的**比较**
- 整数乘法：每位数字相乘(位乘)1次m位和n位整数相乘要做 mn 次**位乘**
- 矩阵相乘：每对**元素乘**1次
 $i \times j$ 矩阵与 $j \times k$ 矩阵相乘要做 ijk 次乘法
- 图的遍历：**置指针**

.....



算法与复杂度

问题及实例

算法

基本运算与规模 输入规模 基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

算法复杂度

- 对于相同输入规模的不同实例，算法的基本运算次数也不一样，可定义两种时间复杂度

最坏情况下的时间复杂度 $W(n)$

算法求解输入规模为 n 的实例所需要的最长时间

- 平均情况下时间复杂度 $A(n)$

在给定同样规模为 n 的输入实例的概率分布下，算法求解这些实例所需要的平均时间



算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子
时间估计

$A(n)$ 计算公式

- 平均情况下时间复杂度 $A(n)$

设 S 是规模为 n 的实例集

实例 $I \in S$ 的概率是 P_I

算法对实例 I 执行的基本运算次数是 t_I

$$A(n) = \sum_{I \in S} P_I t_I$$

在某些情况下可以假定每个输入实例概率相等



● 算法与复杂度

● 问题及实例

● 算法

● 基本运算与规模
输入规模
基本运算

● 复杂度

● 平均复杂度

● 检索例子

顺序检索
最坏情况
平均情况

● 改进检索例子

时间估计

例子：检索

检索问题

输入： 非降顺序排列的数组 L ，元素 n ，数 x

输出： j

若 x 在 L 中， j 是 x 首次出现的下标

否则 $j = 0$

基本运算： x 与 L 中元素的比较



算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

顺序检索算法

$j = 1$, 将 x 与 $L[j]$ 比较.如果 $x = l[j]$,则算法停止, 输出 j ; 如果不等, 则把 j 加1, 继续 x 与 $L[j]$ 的比较, 如果就 $j > n$,则停机并输出0.

实例

1

2

3

4

5

$X = 4$, 需要比较4次

$X = 2.5$, 需要比较5次



最坏情况的时间估计

不同的输入有 $2n+1$ 个，分别对应：

$$x = L[1], x = L[2], \dots, x = L[n]$$

$$X < L[1], X < L[1] < L[2], L[2] < x < L[3], \dots, L[n] < x$$

最坏情况下时间： $W(n) = n$

最坏的输入： x 不在 L 中或 $x = L[n]$ 要做 n 次比较

算法与复杂度

问题及实例

算法

基本运算与规模

输入规模

基本运算

复杂度

平均复杂度

检索例子

顺序检索

最坏情况

平均情况

改进检索例子

时间估计



算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

平均情况时间估计

输入实例的概率分布：

假设 x 在 L 中概率是 p ，且每个位置概率相等

$$\begin{aligned} A(n) &= \sum_{i=1}^n i \frac{p}{n} + (1-p)n \\ &= \frac{p(n+1)}{2} + (1-p)n \end{aligned}$$

当 $p = 1/2$ 时

$$A(n) = \frac{n+1}{4} + n/2 \approx 3n/4$$



改进顺序检索算法

$J = 1$, 将 x 与 $L[j]$ 比较. 如果 $x = L[j]$, 则算法停止, 输出 j ;
如果 $x > L[j]$, 则把 j 加 1, 继续 x 与 $L[j]$ 的比较; 如果 $x < L[j]$, 则停机并输出 0. 如果 $j > n$, 则停机并输出 0.

实例

1	2	3	4	5
---	---	---	---	---

$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 3 次

算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计



算法与复杂度

问题及实例

算法

基本运算与规模
输入规模
基本运算

复杂度

平均复杂度

检索例子

顺序检索
最坏情况
平均情况

改进检索例子

时间估计

时间估计

最坏情况下: $W(n) = n$

平均情况下

输入实例的概率分布: 假设 x 在 L 中每个位置与空隙的概率都相等



改进检索算法平均时间复杂度是多少?

小结:

- 算法最坏何平均情况下的时间复杂度定义
- 如何计算上述时间复杂度



一、算法的复杂性问题

二、算法及其时间复杂度

三、算法的伪码描述

四、函数的渐近的界





算法的伪码描述

算法伪码描述

求最大公约数

运行实例

改进顺序检索

插入排序

运行实例

二分归并排序

算法A

运行实例

小结

- 赋值语句: $\leftarrow$
- 分支语句: if ...then ... [else...]
- 循环语句: while, for, repeat until
- 转向语句: goto
- 输出语句: return
- 调用: 直接写过程的名字
- 注释: //...



● 算法伪码描述

● 求最大公约数

运行实例

● 改进顺序检索

● 插入排序

运行实例

● 二分归并排序

● 算法A

运行实例

● 小结

实例：求最大公约数

算法1.1 Euclid(m, n)

输入：非负整数 m, n ，其中 m 与 n 不全为 0

输出： m 与 n 的最大公约数

1. while $m > 0$ do
2. $r \leftarrow n \bmod m$
3. $n \leftarrow m$
4. $m \leftarrow r$
5. return n

算法伪码描述

求最大公约数

运行实例

改进顺序检索

插入排序

运行实例

二分归并排序

算法A

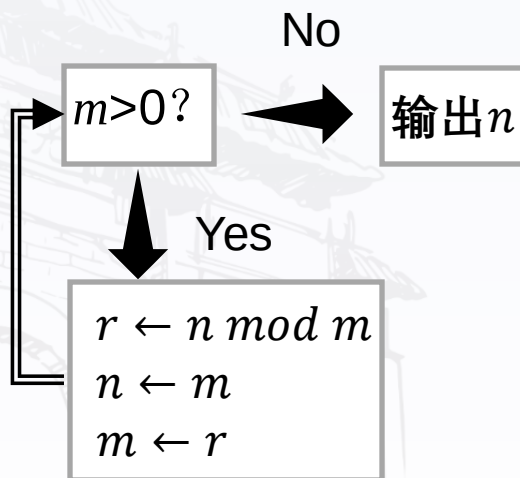
运行实例

小结

运行实例: $n=36, m=15$

while	n	m	r
第1次	36	15	6
第2次	15	6	3
第3次	6	3	0
	3	0	0

输出3





● 算法伪码描述

● 求最大公约数

运行实例

● 改进顺序检索

● 插入排序

运行实例

● 二分归并排序

● 算法A

运行实例

● 小结

实例：改进的顺序检索

算法1.2 Search(L, x)

输入：数组 $L[1..n]$ ，其元素按照从小到大排列，数 x .

输出：若 x 在 L 中，输出 x 的位置下标 j ；否则输出0.

1. $j \leftarrow 1$
2. while $j \leq n$ and $x > L[j]$ do $j \leftarrow j + 1$
3. if $x < L[j]$ or $j > n$ then $j \leftarrow 0$
4. return j



● 算法伪码描述

● 求最大公约数

运行实例

● 改进顺序检索

● 插入排序

运行实例

● 二分归并排序

● 算法A

运行实例

● 小结

实例：插入排序

算法1.2 InsertSort(A, x)

输入： n 个数的数组 A

输出：按照递增顺序排好序的数组 A

1. for $j \leftarrow 2$ to n do
2. $x \leftarrow A[j]$
3. $i \leftarrow j-1$ //3-7行把 $A[j]$ 插入 $A[1..j-1]$
4. while $i > 0$ and $x < A[i]$ do
5. $A[i+1] \leftarrow A[i]$
6. $i \leftarrow i-1$
7. $A[i+1] \leftarrow x$

算法伪码描述

求最大公约数

运行实例

改进顺序检索

插入排序

运行实例

二分归并排序

算法A

运行实例

小结

运行实例

2	4	1	5	3
---	---	---	---	---

$j = 3, x = A[3] = 1$

$i = 2, A[2] = 4$

$i > 0, x < A[2]$

2	4	4	5	3
---	---	---	---	---

$A[3] = 4, i = 1, x = 1$

$i > 0, x < A[1]$

2	2	4	5	3
---	---	---	---	---

$A[2] = 2, i = 0, x = 1$

$i > 0$

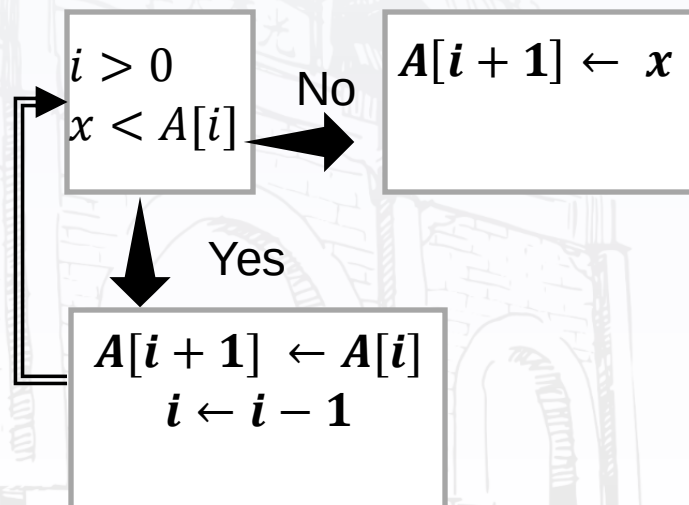
1	2	4	5	3
---	---	---	---	---

4. While $i > 0$ and $x < A[j]$ do

5. $A[i + 1] \leftarrow A[i]$

6. $i \leftarrow i - 1$

7. $A[i + 1] \leftarrow x$





● 算法伪码描述

● 求最大公约数

运行实例

● 改进顺序检索

● 插入排序

运行实例

● 二分归并排序

● 算法A

运行实例

● 小结

例: 二分归并排序

MergeSort(A, p, r) // 归并排序数组 $A[p..r]$,

1. **if $p < r$**

2. **then $q \leftarrow \lfloor (p+r)/2 \rfloor$**

3. **MergeSort(A, p, q)**

4. **MergeSort($A, q+1, r$)**

5. **Merge(A, p, q, r)**

MergeSort有递归调用，也有Merge过程



算法伪码描述

● 求最大公约数

运行实例

● 改进顺序检索

● 插入排序

运行实例

● 二分归并排序

● 算法A

运行实例

● 小结

例: 算法A的伪码

算法A

输入: 实例的数组 $P[0,n]$, 实数 x

输出: y

1. $y \leftarrow P[0]; power \leftarrow 1$

2. *for* $i \leftarrow 1$ *to* n *do*:

3. $power \leftarrow power * x$

4. $y \leftarrow y + P[i] * power$

5. *return* y

这个算法计算的
的是什么?



算法伪码描述

对 $i=1,2,\dots,n$ $\rightarrow$

$power \leftarrow power * x$
 $y \leftarrow y + P[i] * power$

初值

i	power	y
	1	$P[0]$
1	x	$P[0] + P[1] * x$
2	x^2	$P[0] + P[1] * x + P[2] * x^2$
3	x^3	$P[0] + P[1] * x + P[2] * x^2 + P[3] * x^3$
		...

循环

输入 $P[0 \dots n]$ 是 n 次多项式 $P(x)$ 的系数
算法A 计算该多项式在 x 的值



小结

用伪码表示算法

- 伪码不是程序代码，只是给出算法的主要步骤
- 伪码中有哪些关键字
- 伪码中允许过程调用



一、算法的复杂性问题

二、算法及其时间复杂度

三、算法的伪码描述

四、函数的渐近的界





四、函数的渐近的界

01

渐近的界

02

有关定理

03

重要函数



四、函数的渐近的界

01

渐近的界

02

有关定理

03

重要函数

● 渐近的界

● 大O

例子

● 大Ω

例子

● 小o

例子

● 小ω

例子

● Θ

素数

● 小结

大O符号 (渐进上界)

定义： 设 f 和 g 是定义域为自然数集 N 上的函数.若存在正数 c 和 n_0 ,使得对一切 $n \geq n_0$ 有

$$0 \leq f(n) \leq cg(n)$$

成立，则称 $f(n)$ 的渐进的上界是 $g(n)$ ，记作

$$f(n) = O(g(n))$$



渐近的界

- 大O 例子
- 大 Ω 例子
- 小o 例子
- 小 ω 例子
- Θ 素数
- 小结

例子

设 $f(n) = n^2 + n$, 则

$f(n) = O(n^2)$ 取 $c = 2, n_0 = 1$ 即可

$f(n) = O(n^3)$ 取 $c = 1, n_0 = 2$ 即可

1. $f(n) = O(g(n))$, $f(n)$ 的阶不高于 $g(n)$ 的阶
2. 可能存在多个正数 c , 只要指出一个即可
3. 对前面有限个值可以不满足上述不等式
4. 常函数可以写作 $O(1)$



渐近的界

- 大O
- 例子
- 大 Ω
- 例子
- 小o
- 例子
- 小 ω
- 例子
- Θ
- 素数
- 小结

大 Ω 符号 (渐进下界)

定义： 设 f 和 g 是定义域为自然数集 N 上的函数.若存在正数 c 和 n_0 ,使得对一切 $n \geq n_0$ 有

$$0 \leq cg(n) \leq f(n)$$

成立，则称 $f(n)$ 的渐进的下界是 $g(n)$ ，记作

$$f(n) = \Omega(g(n))$$



渐近的界

- 大O 例子
- 大 Ω 例子
- 小o 例子
- 小 ω 例子
- Θ 素数
- 小结

例子

设 $f(n) = n^2 + n$, 则

$f(n) = \Omega(n^2)$ 取 $c = 1, n_0 = 1$ 即可

$f(n) = \Omega(100n)$ 取 $c = 1/100, n_0 = 1$ 即可

1. $f(n) = \Omega(g(n))$, $f(n)$ 的阶 **不低于** $g(n)$ 的阶

2. 可能存在多个正数 c , 只要指出一个即可

3. 对前面有限个值可以不满足上述不等式



渐近的界

- 大O 例子
- 大 Ω 例子
- 小o 例子
- 小 ω 例子
- Θ 素数
- 小结

小o符号 (非渐进紧确上界)

定义： 设 f 和 g 是定义域为自然数集 N 上的函数. 若对于任意正数 c 都存在 n_0 , 使得对一切 $n \geq n_0$ 有

$$0 \leq f(n) < cg(n) \quad (\text{没有了等号})$$

成立, 则记作

$$f(n) = o(g(n))$$



渐近的界

- 大O 例子
- 大Ω 例子
- 小o 例子
- 小ω 例子
- Θ 素数
- 小结

例子

设 $f(n) = n^2 + n$, 则

$f(n) = o(n^3)$ 成立么?

$c \geq 1$ 显然成立, 因为 $n^2 + n < cn^3$ ($n_0 = 2$)
任给 $1 > c > 0$, 取 $n_0 > [2/c]$ 即可. 因为
 $cn \geq cn_0 > 2$ (当 $n \geq n_0$)

$$n^2 + n < 2n^2 < cn^3$$

1. $f(n) = o(g(n))$, $f(n)$ 的阶 **低于** $g(n)$ 的阶.
2. 对于不同正数 c , n_0 不一样. c 越小 n_0 越大.
3. 对前面有限个 n 值可以不满足不等式



渐近的界

- 大O 例子
- 大 Ω 例子
- 小o 例子
- 小 ω 例子
- Θ 素数
- 小结

小 ω 符号 (非渐进紧确下界)

定义： 设 f 和 g 是定义域为自然数集 N 上的函数.若对于任意正数 c 都存在 n_0 ,使得对一切 $n \geq n_0$ 有

$$0 \leq cg(n) < f(n)$$

成立，则记作

$$f(n) = \omega(g(n))$$



渐近的界

- 大O 例子
- 大 Ω 例子
- 小o 例子
- 小 ω 例子
- Θ 素数
- 小结

例子

设 $f(n) = n^2 + n$, 则

$f(n) = \omega(n^2)$ 正确么?

不能写 $f(n) = \omega(n^2)$, 因为取 $c=2$, 不存在 n_0 使得对一切 $n > n_0$, 有下式成立

$$cn^2 = 2n^2 < n^2 + n$$

1. $f(n) = \omega(g(n))$, $f(n)$ 的阶 **高于** $g(n)$ 的阶.
2. 对于不同正数 c , n_0 不等. c 越大 n_0 越大.
3. 对前面有限个 n 值可以不满足不等式



渐近的界

- 大O 例子
- 大 Ω 例子
- 小o 例子
- 小 ω 例子
- Θ 素数
- 小结

Θ 符号 (渐进确界)

定义： 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$,
则记作

$$0 \leq cf(n) = \Theta(f(n))$$

例子: $f(n) = n^2 + n$, $g(n) = 100n^2$, 那么

$f(n) = \Theta(g(n))$ 正确么?

1. $f(n)$ 的阶与 $g(n)$ 的阶相等

2. 对于前面有限个 n 值可以不满足条件.



渐近的界

大O
例子

大 Ω
例子

小o
例子

小 ω
例子

Θ
素数

小结

例子：素数测试

算法 *PrimalityTest*(n)

输入: n , 大于2的奇整数

输出: true or false

```
1.  $s \leftarrow \lfloor n^{\frac{1}{2}} \rfloor$ 
2. for  $j \leftarrow 2$  to  $s$ 
3.   if  $j$  整除  $n$ 
4.   then return false
5. return true
```

问题:

若 $n^{\frac{1}{2}}$ 可在 $O(1)$ 计算,
基本运算是整除, 以下对算法复杂度的表示是否正确?

$$W(n) = O(n^{\frac{1}{2}}) \quad \checkmark$$

$$W(n) = \Theta(n^{\frac{1}{2}}) \quad \times$$



小结

- 五种表示函数的阶的符号
 O o Θ ω Ω
- 定义
- 如何用定义证明函数的阶



四、函数的渐近的界

01

渐近的界

02

有关定理

03

重要函数



有关定理

定理1

证明

估计函数的阶

重要结果

重要结果续

定理2

例子

定理3

小结

有关定理

定理1.1 设 f 和 g 是定义域为自然数集合的函数.

(1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在且等于某个常数 $c > 0$, 那么

$$f(n) = \Theta(g(n)).$$

(2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, 那么 $f(n) = o(g(n))$.

(3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$, 那么 $f(n) = \omega(g(n))$.



● 有关定理

● 定理1

证明

● 估计函数的阶

● 重要结果

● 重要结果续

● 定理2
例子

● 定理3

● 小结

证明定理1.1 (1)

证 根据极限定义, 对于给定的正数 $\varepsilon = c/2$, 存在某个 n_0 , 只要 $n \geq n_0$, 就有

$$\begin{aligned} \left| \frac{f(n)}{g(n)} - c \right| < \varepsilon &\Rightarrow c - \varepsilon < \frac{f(n)}{g(n)} < c + \varepsilon \\ \Rightarrow \frac{c}{2} < \frac{f(n)}{g(n)} < \frac{3c}{2} < 2c \end{aligned}$$

取 $\varepsilon = c/2$

对所有的 $n \geq n_0$, $f(n) \leq 2cg(n)$. 从而推出 $f(n) = O(g(n))$

对所有的 $n \geq n_0$, $f(n) \geq (c/2)g(n)$, 从而推出 $f(n) = \Omega(g(n))$,

于是 $f(n) = \Theta(g(n))$



● 有关定理

● 定理1

证明

● 估计函数的阶

● 重要结果

● 重要结果续

● 定理2
例子

● 定理3

● 小结

例：估计函数的阶

例1 设 $f(n) = \frac{1}{2}n^2 - 3n$, 证明 $f(n) = \Theta(n^2)$.

证 因为

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^2} = \lim_{n \rightarrow \infty} \frac{\frac{1}{2}n^2 - 3n}{n^2} = \frac{1}{2}$$

根据定理1, 有 $f(n) = \Theta(n^2)$



● 有关定理

● 定理1

证明

● 估计函数的阶

● 重要结果

● 重要结果续

● 定理2

例子

● 定理3

● 小结

一些重要结果

可证明：多项式函数的阶低于指数函数的阶

$$n^d = o(r^n), r > 1, d > 0$$

证 不妨设d为正常数,

$$\begin{aligned} \lim_{n \rightarrow \infty} \left(\frac{n^d}{r^n} \right) &= \lim_{n \rightarrow \infty} \left(\frac{dn^{d-1}}{r^n \ln r} \right) = \lim_{n \rightarrow \infty} \left(\frac{d(d-1)n^{d-2}}{r^n (\ln r)^2} \right) \\ &= \dots = \lim_{n \rightarrow \infty} \left(\frac{d!}{r^n (\ln r)^d} \right) = 0 \end{aligned}$$

分子分母
求导数



● 有关定理

● 定理1

证明

● 估计函数的阶

● 重要结果

● 重要结果续

● 定理2
例子

● 定理3

● 小结

一些重要结果 (续)

可证明：对数函数的阶低于幂函数的阶

$$\ln n = o(n^d), d > 0$$

证

$$\begin{aligned} \lim_{n \rightarrow \infty} \left(\frac{\ln n}{n^d} \right) &= \lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n}}{dn^{d-1}} \right) \\ &= \lim_{n \rightarrow \infty} \left(\frac{1}{dn^d} \right) = 0 \end{aligned}$$



有关定理

定理1

证明

估计函数的阶

重要结果

重要结果续

定理2

例子

定理3

小结

定理2

定理 设 f, g, h 是定义域为自然数集合的函数,

(1) 如果 $f=O(g)$ 且 $g=O(h)$, 那么 $f=O(h)$.

(2) 如果 $f=\Omega(g)$ 且 $g=\Omega(h)$, 那么 $f=\Omega(h)$.

(3) 如果 $f=\Theta(g)$ 和 $g=\Theta(h)$, 那么 $f=\Theta(h)$.

函数的阶之间的关系具有传递性



● 有关定理

● 定理1

证明

● 估计函数的阶

● 重要结果

● 重要结果续

● 定理2

例子

● 定理3

● 小结

例子

按照阶从高到低排序以下函数：

$$f(n) = (n^2 + n)/2, g(n) = 10n$$

$$h(n) = 1.5^n, t(n) = n^{\frac{1}{2}}$$

$$h(n) = \omega(f(n)),$$

$$f(n) = \omega(g(n)),$$

$$g(n) = \omega(t(n)),$$

排序 $h(n), f(n), g(n), t(n)$



● 有关定理

● 定理1

证明

● 估计函数的阶

● 重要结果

● 重要结果续

● 定理2
例子

● 定理3

● 小结

定理3

定理 假设 f 和 g 是定义域为自然数集合的函数，若对某个其它的函数 h , 有 $f=O(h)$ 和 $g=O(h)$, 那么

$$f + g = O(h).$$

该性质可以推广到有限个函数.

算法由有限步骤构成. 若每一步的时间复杂度函数的上界都是 $h(n)$, 那么该算法的时间复杂度函数可以写作 $O(h(n))$



有关定理

定理1

证明

估计函数的阶

重要结果

重要结果续

定理2

例子

定理3

小结

小结

- 估计函数的阶的方法
 - 计算极限
 - 阶具有传递性
- 对数函数的阶低于幂函数，多项式函数的阶低于指数函数的阶
- 算法的时间复杂度是各步操作时间之和，在常数步的情况下取最高阶的函数即可



四、函数的渐近的界

01

渐近的界

02

有关定理

03

重要函数



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

基本函数类别

- 阶的高低
- 至少指数级： $2^n, 3^n, n!, \dots$ (不可接受)
- 多项式级： $n, n^2, n \log n, n^{1/2}, \dots$ (可接受)
- 对数多项式级： $\log n, \log^2 n, \log \log n,$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

对数函数

符号：

$$\log n = \log_2 n$$

$$\log^k n = (\log n)^k$$

$$\log \log n = \log(\log n)$$

性质：

$$(1) \log_k n = \Theta(\log_l n)$$

$$(2) \log_b n = o(n^\alpha) \quad \alpha > 0$$

$$(3) a^{\log_b n} = n^{\log_b a}$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

有关性质 (1) 的证明

$$\log_k n = \frac{\log_l n}{\log_l k} \quad \log_l k \text{ 为常数 (换底公式)}$$

$$\lim_{n \rightarrow \infty} \frac{\log_k n}{\log_l n} = \lim_{n \rightarrow \infty} \frac{\log_l n}{\log_l k \cdot \log_l n} = \frac{1}{\log_l k}$$

根据定理, $\log_k n = \Theta(\log_l n)$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

有关性质 (2) (3) 的证明

$$\log_b n = \Theta(\ln n)$$

$$\ln n = o(n^\alpha)$$



$$\log_b n = o(n^\alpha) \quad \alpha > 0$$

$$a^{\log_b n} = n^{\log_b a}$$



$$\log_b n \log_b a = \log_b a \log_b n$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

指数函数与阶乘

Stirling公式
$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n)$$

$$n! = \Omega(2^n)$$

$$\log(n!) = \Theta(n \log n)$$



应用：估计搜索空间大小

m元钱、投资n个
项目的分配方案数

$$C_{m+n-1}^m = \frac{(m+n-1)!}{m!(n-1)!}$$

$$\begin{aligned} &= \frac{\sqrt{2\pi(m+n-1)}(m+n-1)^{m+n-1} \left(1 + \Theta\left(\frac{1}{m+n-1}\right)\right)}{\sqrt{2\pi m} m^m \left(1 + \Theta\left(\frac{1}{m}\right)\right) \sqrt{2\pi(n-1)}(n-1)^{n-1} \left(1 + \Theta\left(\frac{1}{n-1}\right)\right)} \\ &= \Theta((1+\epsilon)^{m+n-1}) \end{aligned}$$

重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

下界 上界

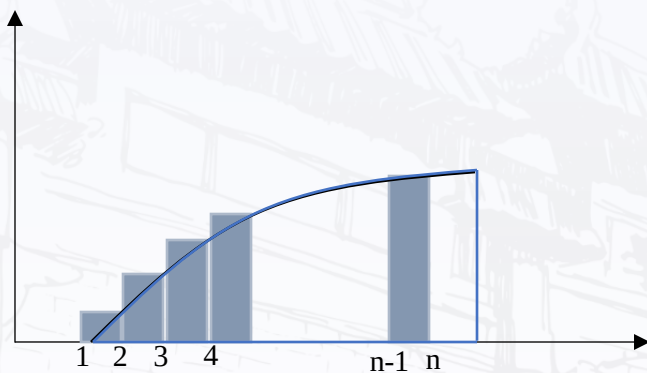
取整函数

取整函数性质

证明

实例：阶排序

$\log(n!) = O(n \log n)$ 的证明(下界)



$$\begin{aligned}\log(n!) &= \sum_{k=1}^n \log k \geq \int_1^n \log x \, dx \\ &= \log e (n \ln n - n + 1) = \Omega(n \log n)\end{aligned}$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

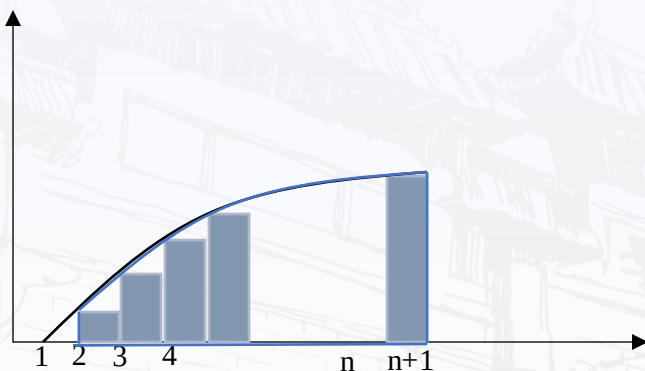
取整函数

取整函数性质

证明

实例：阶排序

$\log(n!) = O(n \log n)$ 的证明(上界)



$$\begin{aligned}\log(n!) &= \sum_{k=1}^n \log k \leq \int_2^{n+1} \log x \, dx \\ &= O(n \log n)\end{aligned}$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

取整函数

取整函数的定义

$\lfloor x \rfloor$: 表示小于等于 x 的最大的整数

$\lceil x \rceil$: 表示大于等于 x 的最小的整数

实例

$$\lfloor 2.6 \rfloor = 2$$

$$\lceil 2.6 \rceil = 3$$

$$\lfloor 2 \rfloor = \lceil 2 \rceil = 2$$

应用：二分检索

输入数组长度： n

中位数的位置 $\lfloor n/2 \rfloor$

与中位数比较后子问题大小： $\lfloor n/2 \rfloor$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

取整函数的性质

取整函数具有下述性质：

$$(1) \quad x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$$

$$(2) \quad \lfloor x+n \rfloor = \lfloor x \rfloor + n, \quad \lceil x+n \rceil = \lceil x \rceil + n, \quad \text{其中 } n \text{ 为整数}$$

$$(3) \quad \left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil = n \quad \text{其中 } n \text{ 为整数}$$

$$(4) \quad \left\lceil \frac{\left\lfloor \frac{n}{a} \right\rfloor}{b} \right\rceil = \left\lfloor \frac{n}{ab} \right\rfloor, \quad \left\lfloor \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rfloor = \left\lceil \frac{n}{ab} \right\rceil \quad \text{其中 } n, a, b \text{ 为整数}$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

证明 (1)

(1) 如果 x 是整数 n ，根据定义 $\lfloor x \rfloor = \lceil x \rceil = n$ ，
$$x-1 < \lfloor x \rfloor = x = \lceil x \rceil < x+1$$

如果 $n < x < n+1$, n 为整数，那么
$$\lfloor x \rfloor = n, \lceil x \rceil < n+1.$$

从而有

$$x-1 < n = \lfloor x \rfloor, n < x < n+1 = \lceil x \rceil$$

$$\rightarrow x-1 < n = \lfloor x \rfloor, n < x < n+1 = \lceil x \rceil$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

按照阶排序

$$\log^2 n, \quad 1, \quad n!, \quad n2^n, \quad n^{1/\log n},$$

$$(3/2)^n, \quad \sqrt{\log n}, \quad (\log n)^{\log n},$$

$$2^{2^n}, \quad n^{\log \log n}, \quad n^3, \quad \log \log n,$$

$$n \log n, \quad n, \quad 2^{\log n}, \quad \log n, \quad \log(n!)$$



重要函数

基本函数

对数函数

性质1证明

性质2,3证明

指数函数阶乘

应用

上界 下界

取整函数

取整函数性质

证明

实例：阶排序

按照阶排序 (结果)

$$2^{2^n}, \quad n!, \quad n2^n, \quad (3/2)^n, \quad (\log n)^{\log n} = n^{\log \log n},$$

$$n^3, \quad \log(n!) = \Theta(n \log n), \quad n = \Theta(2^{\log n}),$$

$$\log^2 n, \quad \log n, \quad \sqrt{\log n}, \quad \log \log n,$$

$$n^{1/\log n} = \Theta(1)$$



一、算法的复杂性问题

二、算法及其时间复杂度

三、算法的伪码描述

四、函数的渐近的界





作业：截至时间**3月19日**前上传**QQ作业**

定理1.1 设 f 和 g 是定义域为自然数集合的函数.

(1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在且等于某个常数 $c > 0$, 那么

$$f(n) = \Theta(g(n)).$$

(2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, 那么 $f(n) = o(g(n))$.

(3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$, 那么 $f(n) = \omega(g(n))$.